Now we can answer the question: Why does *Analog Devices* sell *Digital Signal Processors*? Only a decade ago, state-of-the-art signal processing was carried out with precision op amps and similar transistor circuits. Today, the highest quality *analog* processing is accomplished with *digital* techniques. Analog Devices is a great role-model for individuals and other companies; hold on to your vision and goals, but don't be afraid to adapt with the changing technology!

Another Look at Fixed versus Floating Point

In this last example, we took advantage of one of the SHARC DSP's key features, its ability to handle floating point calculations. Even though the samples are in a fixed point format when passed to and from the codec, we go to the trouble of converting them to floating point for the intermediate FIR filtering algorithm. As discussed in the last chapter, there are two reasons for wanting to process the data with floating point math: *ease of programming*, and *performance*. Does it really make a difference?

For the programmer, yes, it makes a large difference. Floating point code is far easier to write. Look back at the assembly program in Table 29-2. There are only two lines (41 and 42) in the main FIR filter. In contrast, the fixed point programmer must add code to manage the data at each math calculation. To avoid overflow and underflow, the values must be checked for size and, if needed, scaled accordingly. The intermediate results will also need to be stored in an extended precision accumulator to avoid the devastating effects of repeated round-off error.

The issue of performance is much more subtle. For example, Fig. 29-5a shows an FIR low-pass filter with a moderately sharp cutoff, as described in Chapter 16. This "large scale" curve would look the same whether fixed or floating point were used in the calculation. To see the difference between these two methods, we must zoom in on the amplitude by a factor of several hundred as shown in (b), (c), and (d). Here we can see a clear difference. The floating point execution, (b), has such low round-off noise that its performance is limited by the way we designed the filter kernel. The 0.02% overshoot near the transition is a characteristic of the Blackman window used in this filter. The point is, if we want to improve the performance, we need to work on the *algorithm*, not the *implementation*. The curves in (c) and (d) show the round-off noise introduced when each point in the filter kernel is represented by 16 and 14 bits, respectively. A better algorithm would do nothing to make these better curves; the shape of the actual frequency response is swamped by noise.

Figure 29-6 shows the difference between fixed and floating point in the *time domain*. Figure (a) shows a wiggly signal that exponentially decreases in amplitude. This might represent, for example, the sound wave from a plucked string, or the shaking of the ground from a distant explosion. As before, this "large scale" waveform would look the same whether fixed or floating point were used to represent the samples. To see the difference,